# AlgoBox

Logiciel pédagogique d'aide à la création et à l'exécution d'algorithmes

## Manuel de l'utilisateur (version 0.7.2)

## Sommaire


- 1. Fonctionnement général
- 2. Première prise en main d'AlgoBox avec un exemple très simple
- 3. Autres règles d'utilisation d'AlgoBox
  - 3.1 Les types de variables
  - 3.2 Les expressions conditionnelles
  - 3.3 Les boucles
  - 3.4 Lecture et affichage des variables
  - 3.5 Affichage d'une variable ou d'un message
  - 3.6 Utilisation d'une fonction numérique
  - 3.7 Tracer des points et des segments dans un repère
  - 3.8 Utilisation d'une fonction récursive
  - 3.9 Copier, coller et couper une ligne ou bloc de l'algorithme
  - 3.10 Ajout de commentaires
  - 3.11 Provoquer un arrêt temporaire de l'exécution d'un algorithme avec l'instruction *PAUSE*
  - 3.12 Exécution d'un algorithme en mode "pas à pas"
  - 3.13 Calculs numériques et arrondis
  - 3.14 Export des résultats d'un algorithme en pdf
- 4. Raccourcis clavier
- 5. Tutoriel d'initiation à l'algorithmique avec AlgoBox
- 6. Déboguer un algorithme
- 7. Sécurités lors de l'exécution d'un algorithme
- 8. Augmenter l'espace disponible pour la conception d'un algorithme
- 9. Simulation de tableaux à deux dimensions
- 10. Edition du code en mode "éditeur de texte"
- 11. Le menu "Extension"


# 1. Fonctionnement général

Le fonctionnement d'AlgoBox est basé sur les deux principes suivants :

1. Pour pouvoir utiliser une variable dans son algorithme, il faut d'abord la déclarer en utilisant le bouton Déclarer nouvelle variable qui est activable à tout moment.
2. Pour ajouter un nouvel élément à l'algorithme (autre que la déclaration d'une variable), il faut d'abord insérer une nouvelle ligne en utilisant le bouton Nouvelle ligne (raccourci-clavier : Ctrl+Return). Ce bouton n'est activable que s'il est possible d'ajouter une nouvelle ligne après la position actuelle dans le code. Par exemple, l'ajout d'une nouvelle ligne après "Fin_Algorithme" n'est pas autorisé.
   Avant d'insérer une nouvelle ligne, il peut donc être nécessaire de se positionner à l'endroit voulu dans la code à l'aide de la souris ou des touches "flèches" du clavier.
   Une fois que l'on est positionné sur une nouvelle ligne, il suffit de cliquer sur l'un des boutons disponibles dans le panneau inférieur pour insérer l'instruction (ou le bloc d'instructions) désiré.

Autres caractéristiques du logiciel :

- Après avoir entré certaines instructions, il est possible d'en modifier les paramètres en cliquant sur le bouton

Modifier Ligne (le bouton n'est activable que lorsque les changements sont possibles).
Pour les autres instructions, notamment pour la déclaration de variables, la seule possibilité est de supprimer la ligne qui ne correspond plus au code désiré.

- Il est possible à tout moment d'effacer une instruction avec le bouton Supprimer Ligne/Bloc
  **Attention : la suppression d'une ligne "SI ALORS", "POUR...DE...A" et "TANT QUE" entraîne la suppression de tout le bloc d'instructions qui en dépend** (tout ce qui est contenu entre "DEBUT..." et "FIN...")
- Pour tester le fonctionnement de l'algorithme, il suffit de cliquer sur le bouton Tester (dans la barre d'outils ou dans le panneau de droite). Une fenêtre apparaît alors dans laquelle il suffit de cliquer sur le bouton Lancer l'algorithme pour précéder à l'exécution de l'algorithme. S'il n'y a pas d'erreurs de syntaxe détectée d'entrée dans le code de l'algorithme, les résultats apparaissent entre les messages "***Algorithme lancé***" et "***Algorithme terminé***".
  **L'exécution d'un algorithme peut-être stoppé à tout moment en cliquant sur le bouton** Arrêter.
  La détection d'une erreur de calcul (division par zéro, par exemple) entraine l'arrêt immédiat de l'exécution d'un algorithme.
  Il est aussi possible d'imprimer la partie supérieure de la fenêtre de test contenant le code et l'éventuel graphique à l'aide d'un clic-droit sur cette zone.
- Les opérations standards d'ouverture et d'enregistrement sont disponibles via la barre d'outils et le menu "Fichier". Il est conseillé avant de sauvegarder son travail de remplir le champ "Présentation de l'algorithme" dans lequel on peut présenter brièvement l'algorithme sur lequel on travaille.
- Il est aussi possible d'imprimer et d'exporter le code de l'algorithme via le menu "Fichier".
- L'option `"Exporter algorithme complet vers page web"` du menu `"Fichier"` permet d'exécuter un algorithme déjà conçu à partir d'une simple page web que l'on peut aisément alors transférer sur un site internet.
  **Limitations importantes :**
  - pour les algorithmes utilisant le repère graphique, il faut utiliser un autre navigateur qu'internet explorer ou rester dans AlgoBox (internet explorer, qui n'inclut pas certaines fonctionnalités, ne peut en effet pas afficher les graphiques)
  - certaines fonctionnalités d'AlgoBox, comme le mode "pas à pas", l'affichage progressif des données ou la possibilité d'arrêter à tout moment l'exécution d'un algorithme, ne sont plus disponibles pour les algorithmes enregistrés sous forme de simple page web.
- Une série d'exemples d'algorithmes est fournie avec le logiciel. On peut y accéder à l'aide de l'option `"Ouvrir exemples"` du menu `"Ficher"`. *(les exemples dont le nom commence par "prof" servent à illustrer les capacités d'AlgoBox et ne sont pas destinés à être utilisés tels quels en classe)*

# 2. Première prise en main d'AlgoBox avec un exemple très simple

Supposons que l'on veuille calculer l'image de n'importe quel réel x par la fonction f définie par f(x)=3x+4.

1. On déclare d'abord la variable x en cliquant sur Déclarer nouvelle variable. On entre alors x dans le champ Nom de la variable, on s'assure que le type NOMBRE est bien sélectionné et on clique sur "OK".
2. On effectue la même opération avec la variable y.
3. On s'assure que la ligne courante dans le code est bien positionnée sur "DEBUT_ALGORITHME" et on clique alors sur Nouvelle Ligne.
4. Pour pouvoir lire la valeur de x, on clique ensuite sur Ajouter LIRE variable et on s'assure que la variable x est bien sélectionnée avant de cliquer sur "OK".
5. La ligne courante étant bien "LIRE x", on clique sur le bouton Nouvelle Ligne, puis sur le bouton AFFECTER valeur à variable. On sélectionne alors y en face de "La variable" et on entre 3*x+4 dans le champ après "prend la valeur" avant de cliquer sur "OK".
6. Reste à afficher la valeur de y. Pour cela on ajoute une nouvelle ligne après "y PREND_LA_VALEUR 3*x+4" (clic sur le bouton Nouvelle Ligne). On clique alors sur le bouton Ajouter AFFICHER variable et on sélectionne y après "Afficher la variable" avant de cliquer sur "OK".
7. Il n'y a plus qu'à tester l'algorithme en cliquant sur le bouton Tester. Dans la fenêtre qui s'affiche, il suffit alors de cliquer sur le bouton Lancer l'algorithme. On entre alors la valeur de x : son image par la fonction f apparaît alors dans la zone "Résultat". Pour calculer une autre image, il suffit de cliquer à nouveau sur Lancer algorithme. Pour revenir à la fenêtre principale d'AlgoBox, cliquez sur le bouton Fermer.

# 3. Autres règles d'utilisation d'AlgoBox

1. **Les types de variables**:
   AlgoBox permet l'utilisation de trois types de variables : les nombres (type `NOMBRE`), les listes de nombres (type `LISTE`) et les chaînes de caractères (type `CHAINE`).

   **Opérations avec les nombres :**
   En plus des 4 opérations de base (+,-,*,/), les calculs possibles sont (`x` étant une variable du type `NOMBRE`) :
   - Racine carrée d'une variable x : `sqrt(x)`
   - Mettre une variable x à la puissance n : `pow(x,n)`
   - Obtenir un nombre pseudo-aléatoire compris en 0 et 1 : `random()`
   - Nombre PI : `Math.PI`
   - Partie entière d'une variable x : `floor(x)`
   - Cosinus d'une variable x (en radians): `cos(x)`
   - Sinus d'une variable x (en radians): `sin(x)`
   - Tangente d'une variable x (en radians): `tan(x)`
   - Exponentielle d'une variable x : `exp(x)`
   - Logarithme népérien d'une variable x : `log(x)`
   - Valeur absolue d'une variable x : `abs(x)`
   - Arrondi d'une variable x à l'entier le plus proche : `round(x)`
   - Reste de la division de la variable x par la variable y : `x%y`
   - ArcCosinus d'une variable x : `acos(x)`
   - ArcSinus d'une variable x : `asin(x)`
   - ArcTangente d'une variable x : `atan(x)`

   Il faut utiliser le . comme séparateur décimal.

   **Fonctions supplémentaires :**
   - `ALGOBOX_ALEA_ENT(p,n)` : renvoie un entier pseudo-aléatoire compris entre `p` et `n`.
   - `ALGOBOX_COEFF_BINOMIAL(n,p)` (ou `ALGOBOX_NB_COMBINAISONS(n,p)`) : renvoie le coefficient binomial "`p` parmi `n`".
   - `ALGOBOX_LOI_BINOMIALE(n,p,k)` : renvoie la probabilité d'obtenir `k` succès en répétant `n` fois une expérience aléatoire dont la probabilité d'obtenir un succès est `p` avec le modèle de la loi binomiale.
   - `ALGOBOX_LOI_NORMALE_CR(x)` : renvoie p(X<x) pour la loi normale centrée réduite
   - `ALGOBOX_LOI_NORMALE(esp,ecart,x)` : renvoie p(X<x) pour la loi normale d'espérance `esp` et d'écart-type `ecart`.
   - `ALGOBOX_INVERSE_LOI_NORMALE_CR(p)` : opération inverse de `ALGOBOX_LOI_NORMALE_CR(x)`.
   - `ALGOBOX_INVERSE_LOI_NORMALE(esp,ecart,p)` : opération inverse de `ALGOBOX_LOI_NORMALE(esp,ecart,x)`.
   - `ALGOBOX_FACTORIELLE(n)` : renvoie la factorielle de l'entier `n` (pour n<70)

   **Opérations avec les listes :**
   - Les listes AlgoBox sont des listes numérotées de nombres.
   - Si vous sélectionnez pour *la variable* une variable du type `LISTE`, vous devez indiquer dans le champ *rang du terme de la liste* le numéro du terme de la liste auquel vous voulez affecter une valeur.
   - Pour remplir directement une liste, il faut indiquer dans le champ *rang du terme de la liste* le rang du premier terme de la liste et séparer ensuite les valeurs par le caractère `:` dans le champ *prend la valeur*. (on obtient alors une instruction du style `maliste[1]` *prend la valeur* `1:2:3:4` )
   - Pour utiliser un terme d'une liste dans un calcul, il faut utiliser la syntaxe suivante : `nomliste[rang]`
     Exemple : `moy` *prend la valeur* `(maliste[1]+maliste[2]+maliste[3])/3` (la variable du type `NOMBRE` `moy` contiendra la moyenne des trois premiers termes de la liste `maliste`)
   - Les calculs possibles avec une liste sont (tous les résultats sont du type NOMBRE):
     - Somme : `ALGOBOX_SOMME(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
     - Moyenne : `ALGOBOX_MOYENNE(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
     - Variance : `ALGOBOX_VARIANCE(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
     - Ecart-type : `ALGOBOX_ECART_TYPE(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
     - Médiane : `ALGOBOX_MEDIANE(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
     - Premier quartile : `ALGOBOX_QUARTILE1((nom_de_la_liste,rang_premier_terme,rang_dernier_terme)` (définition calculatrice : médiane de la sous-série inférieure - la série doit comporter au moins 4 valeurs, sinon une erreur est déclarée)

- Troisième quartile :
  `ALGOBOX_QUARTILE3((nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
  (définition calculatrice : médiane de la sous-série supérieure - la série doit comporter au moins 4 valeurs, sinon une erreur est déclarée)
- Premier quartile Bis :
  `ALGOBOX_QUARTILE1_BIS((nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
  (autre définition : plus petite valeur telle qu'au moins 25% des données lui soient inférieures - la série doit comporter au moins 4 valeurs, sinon une erreur est déclarée)
- Troisième quartile Bis :
  `ALGOBOX_QUARTILE3_BIS((nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
  (autre définition : plus petite valeur telle qu'au moins 75% des données lui soient inférieures - la série doit comporter au moins 4 valeurs, sinon une erreur est déclarée)
- Minimum :
  `ALGOBOX_MINIMUM(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
- Maximum :
  `ALGOBOX_MAXIMUM(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
- Rang du minimum :
  `ALGOBOX_POS_MINIMUM(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`
- Rang du maximum :
  `ALGOBOX_POS_MAXIMUM(nom_de_la_liste,rang_premier_terme,rang_dernier_terme)`

**Opérations avec les chaînes :**

- Le contenu d'une chaîne doit être encadré par des guillemets :
  Exemple : `a` *prend la valeur* `"bonjour"` (`a` étant une variable du type `chaine`)
- Il est possible d'ajouter (concaténer) des chaînes :
  Exemple : `b` *prend la valeur* `a+"bonjour"` (`a` et `b` étant des variables du type `CHAINE`)
- Il est possible d'extraire le contenu d'une chaîne avec l'instruction
  `chaîne.substr(position_premier_caractère_à_extraire,nombre_de_caractères_à_extraire)`.
  Attention : la premier caractère a pour position 0 (et pas 1)
  Exemple : `b` *prend la valeur* `a.substr(4,2)` (`b` sera alors formé des 5ème et 6ème caractères de `a` ; `a` et `b` étant des variables du type `CHAINE`)
- Un nombre peut-être transformé en chaîne avec l'instruction `nombre.toString()`
  Exemple : `machaine` *prend la valeur* `nb.toString()` (`nb` étant une variable du type `NOMBRE` et `machaine` étant une variable du type `CHAINE`)
- La longueur d'une chaine peut-être obtenue avec l'instruction `chaine.length`
  Exemple : `longueur` *prend la valeur* `machaine.length` (`longueur` étant une variable du type `NOMBRE` et `machaine` étant une variable du type `CHAINE`)
- L'instruction `machaine.charCodeAt(pos)` permet d'obtenir le nombre égal au code ascii de la lettre figurant à la position `pos` dans la chaine `machaine` (Attention : le premier caractère a pour position 0).
- Inversement, l'instruction `String.fromCharCode(nombre)` renvoie une chaine contenant le caractère dont le code ascii est égal à `nombre`.

2. **Les expressions conditionnelles** (structures `SI ALORS` et `TANT QUE`) :
   **Exemples de syntaxe pour la condition :**
   - Pour vérifier si x est égal à 2, la condition à écrire est : `x==2`
   - Pour vérifier si x est différent de 2, la condition à écrire est : `x!=2`
   - Pour vérifier si x est strictement inférieur à 2, la condition à écrire est : `x<2`
   - Pour vérifier si x est inférieur ou égal à 2, la condition à écrire est : `x<=2`
   - Pour vérifier si x est strictement supérieur à 2, la condition à écrire est : `x>2`
   - Pour vérifier si x est supérieur ou égal à 2, la condition à écrire est : `x>=2`

   **Il est possible de combiner plusieurs conditions avec ET et OU :**
   - La condition à écrire pour vérifier que x est strictement compris entre 1 et 5 est : `x>1 ET x<5`
   - La condition à écrire pour vérifier que x est égal à 3 OU à 5 est : `x==3 OU x==5`

   Il est aussi possible d'utiliser des calculs dans les expressions conditionnelles. Exemple de condition : `x<sqrt(n)`
   **Avertissement :** le nombre d'itérations pour les boucles `TANT QUE` est limité à 500000 pour des raisons de sécurité.
3. **Les boucles** (structure `POUR ALLANT DE A`)
   - La variable utilisée pour la boucle (champ *POUR la variable*) ne peut-être que du type `NOMBRE` et la valeur de cette variable est automatiquement augmentée de 1 à chaque boucle.
   - Les champs *ALLANT DE* et *A* peuvent contenir des nombres ou des variables du type `NOMBRE`. C'est à l'utilisateur de vérifier que la valeur du champ *ALLANT DE* est bien inférieure à celle du champ *A*.
   - **Avertissement :** le nombre d'itérations des boucles est limité à 500000 pour des raisons de sécurité.

4. **Lecture et affichage des variables** (commandes `LIRE variable` et `AFFICHER variable`)
   Si on veut lire ou afficher le terme d'une liste, il faut sélectionner le nom de la liste et obligatoirement renseigner le champ *Rang du terme de la liste* qui peut contenir un nombre ou le nom d'une variable du type `NOMBRE`.
5. **Affichage d'une variable ou d'un message** (commandes `AFFICHER variable` et `AFFICHER message`)
   Si la case *Ajouter un retour à la ligne* est cochée, le prochain affichage d'une variable ou d'un message se fera sur une nouvelle ligne. Dans le cas contraire, le prochain affichage se fera directement à la suite du précédent.
6. **Utilisation d'une fonction numérique**
   En activant l'option Utiliser une fonction dans l'onglet "Utiliser une fonction numérique", on peut utiliser l'image de n'importe quel nombre (ou variable de type nombre) par la fonction notée `F1` dans le code de l'algorithme. Il suffit pour cela d'entrer l'expression de `F1(x)` en fonction de `x` dans le champ prévu pour cela.
   Pour utiliser l'image d'un nombre `nb` par la fonction `F1` dans l'algorithme, il suffit d'utiliser le code : `F1(nb)` (cela peut se faire dans une affectation ou dans une expression conditionnelle).
7. **Tracer des points et des segments dans un repère**
   En activant l'option Utiliser un repère dans l'onglet "Dessiner dans un repère", un repère graphique est automatiquement ajouté dans la fenêtre de test de l'algorithme. Il est alors possible d'inclure dans le code de l'algorithme des instructions pour tracer des points et des segments dans ce repère en utilisant les boutons Ajouter TRACER POINT et Ajouter TRACER SEGMENT .
8. **Utiliser une fonction récursive**
   En activant l'option Utiliser la fonction F2... dans l'onglet "Fonction avancée", il est possible de définir une fonction récursive (ou une fonction définie par "morceaux") dépendant de un ou plusieurs paramètres.
   Pour cela, il faut d'abord renseigner le champ paramètres en séparant le nom des variables par une virgule.
   Ensuite, il faut ajouter un par un les cas à l'aide du bouton Ajouter : tous les cas s'affichent au fur et à mesure dans le panneau de droite (pour modifier un cas, il suffit de "double-cliquer" dessus).
   Pour finir, on renseigne le champ Dans les autres cas, renvoyer.
   (l'exemple prof_pgcd_recursif.alg fourni avec le programme peut permettre de mieux comprendre le fonctionnement de cette fonction F2).
9. **Copier, coller et couper une ligne ou un bloc de l'algorithme**
   Via le menu Edition, il est possible de copier/coller/couper :
   - une ligne de code du type ...PREND_LA_VALEUR..., AFFICHER.... et TRACER....
   - un bloc du type POUR ALLANT DE A, SI ALORS et TANT QUE. Pour copier et couper tout le contenu de ce type de bloc, il faut se placer sur la première ligne du bloc en question.

   Remarque : pour **coller** une ligne ou un bloc de code, **il faut d'abord créer une nouvelle ligne** conformément aux règles d'édition en vigueur sur AlgoBox.
10. **Ajout de commentaires**
    L'ajout d'un commentaire dans le code de l'algorithme se fait une ligne vierge grâce au bouton Commentaire. Tout commentaire apparaît en italique et est précédé des caractères `//`.
11. **Provoquer un arrêt temporaire de l'exécution d'un algorithme avec l'instruction *PAUSE***
    L'instruction PAUSE arrête l'exécution de l'algorithme. L'utilisateur peut alors en cliquant sur les boutons Continuer et Arrêter décider de poursuivre ou de mettre définitivement fin à l'exécution de l'algorithme.
12. **Exécution d'un algorithme en mode "pas à pas"**
    - Pour lancer le mode "pas à pas", il suffit de cocher la case correspondante avant de cliquer sur le bouton Lancer Algorithme.
    - La ligne de l'algorithme correspondante à chaque arrêt est signalée en rouge.
    - L'entrée et la sortie de chaque bloc SI, SINON, TANT QUE et POUR est systématiquement signalée.
    - Après chaque instruction pouvant modifier la valeur d'une variable (LIRE, AFFECTER, DEBUT_POUR), l'état de l'ensemble de toutes les variables est affichée (les listes sont affichées sur une ligne séparée).
    - Lors de chaque arrêt, l'utilisateur peut en cliquant sur les boutons Continuer et Arrêter décider de poursuivre ou de mettre définitivement fin à l'exécution de l'algorithme. Pour poursuivre l'exécution en mode "pas à pas" on peut aussi utiliser la touche "flèche bas" du clavier.
13. **Calculs numériques et arrondis**
    Dans AlgoBox, comme avec tous les langages de programmation, la représentation interne des nombres qui ne sont ni des entiers, ni des décimaux simples (du style 0.05) peut engendrer des problèmes de précision et d'arrondis. Une égalité mathématique peut donc se retrouver non vérifiée lors de l'exécution réelle d'un algorithme.
    **L'utilisateur doit être conscient de ces problèmes de précision dès qu'il manipule des nombres non entiers.** La prudence est notamment de mise lors de l'utilisation d'un bloc `TANT_QUE` : une condition ne prenant pas en compte les problèmes de précision en informatique peut engendrer une boucle infinie (qui sera stoppée par le mécanisme de sécurité interne d'AlgoBox). Il convient donc de tenir compte de ces limites de précision

lors de la conception de certains algorithmes.

14. **Export des résultats d'un algorithme en pdf**
    Lors de l'exécution d'un algorithme, il est possible d'exporter en pdf l'ensemble des résultats d'un algorithme (sorties, graphique, code de l'algorithme) via le menu contextuel qui s'affiche en effectuant un clic-droit sur la partie supérieure de la fenêtre de test.

# 4. Raccourcis clavier

- Menu "Fichier" :
  - Nouveau : Ctrl+N
  - Ouvrir : Ctrl+O
  - Sauver : Ctrl+S
  - Quitter : Ctrl+Q
- Menu "Edition" :
  - Copier ligne : Ctrl+C
  - Coller ligne : Ctrl+V
  - Couper ligne : Ctrl+X
- Mise au point de l'algorithme :
  - Nouvelle ligne : Ctrl+Entrée
  - Déclarer nouvelle variable : Ctrl+D
  - Tester algorithme : Ctrl+T
  - Lire variable : Ctrl+L
  - Affecter valeur à variable : Ctrl+F
  - Afficher variable : Ctrl+R
  - Afficher message : Ctrl+M
  - Si...Alors : Ctrl+I
  - Pour...De...A : Ctrl+P
  - Tant...Que : Ctrl+U
- Exécution de l'algorithme :
  - Lancer Algorithme : Entrée
  - Continuer : Flèche Bas
  - Arrêter : Echap
  - Lire variable : Ctrl+Q

# 5. Tutoriel d'initiation à l'algorithmique avec AlgoBox

Un module en ligne d'initiation aux principes de base de l'algorithmique avec AlgoBox peut-être lancé via l'option `Initiation en ligne à l'algorithmique` du menu `Tutoriel`.

# 6. Déboguer un algorithme

AlgoBox n'inclut pas de débogueur complet intégré et infaillible. Toutefois,

- lors de l'arrêt de l'exécution d'un algorithme suite à une erreur, la dernière ligne exécutée est signalée automatiquement;
- une erreur de calcul entraîne automatiquement l'arrêt de l'exécution : la ligne où figure la calcul est signalée;
- pour aider à déboguer un algorithme qui ne donne pas les résultats attendus :
  - il est possible d'afficher à tout moment l'état d'une variable (instruction AFFICHER Variable) et d'inclure une instruction PAUSE;
  - l'exécution de l'algorithme en mode "pas à pas" permet d'afficher étape pas étape la valeur de toutes les variables.

Remarque : si l'algorithme ne se lance pas, c'est que le code contient une grave erreur de syntaxe empêchant son démarrage. L'utilisateur est alors invité à vérifier notamment la syntaxe des affectations et des conditions (ainsi que les éventuels tracés graphiques et la définition de la fonction F1)

# 7. Sécurités lors de l'exécution d'un algorithme

Toute activité de programmation comporte des risques. Pour limiter ces risques, AlgoBox dispose de 4 sécurités lors de l'exécution d'un algorithme (afin d'empêcher notamment les boucles infinies) :

- Une boucle seule ne peut aller au delà de 500 000 itérations;
- Le nombre global d'itérations pour tout l'algorithme ne peut dépasser 5 millions;
- Le nombre d'affichages consécutifs est limité à 1000 : au-delà, l'utilisateur doit confirmer sa volonté de continuer à afficher des données;
- L'utilisateur peut arrêter l'exécution d'un algorithme à tout moment.
- Le nombre de récursions pour la fonction "avancée" F2 est limité à 100 000.

# 8. Augmenter l'espace disponible pour la conception d'un algorithme

Afin d'augmenter la hauteur du panneau contenant le code de l'algorithme, il est possible :

- de cacher la barre d'outils et le cadre de présentation de l'algorithme via le menu Affichage
- de replier un bloc de code en cliquant sur le "triangle" correspondant (un autre clic sur ce même triangle permet de déplier le bloc)

# 9. Simulation de tableaux à deux dimensions

Pour utiliser un tableau dans AlgoBox :

- On se sert d'une variable de type `LISTE`. Exemple : `montableau est du type LISTE`
- Pour affecter une valeur à l'élément du tableau correspondant à la ligne `li` et à la colonne `col`, il suffit de remplir le champ *Rang du terme de la liste* par : `li*(nombre de colonnes)+col`. On obtient alors une ligne de la forme : `montableau[li*(nombre de colonnes)+col] PREND_LA_VALEUR...`
- Pour réutiliser la valeur d'un élément du tableau dans un calcul, on utilise la syntaxe : `montableau[li*(nombre de colonnes)+col]`.

# 10. Edition du code en mode "éditeur de texte"

Via l'option `"Mode Edition"` du menu `"Edition"`, il est possible de passer du mode d'édition du code par défaut (avec boîtes de dialogue) à un mode d'édition du code comme sur un éditeur de texte classique. Ce mode d'édition, **à réserver aux utilisateurs expérimentés**, répond aux caractéristiques suivantes :

- **Une ligne ne peut contenir qu'une instruction** et chaque ligne doit correspondre exactement à une branche de l'arbre utilisé dans le mode d'édition "normal". Ainsi,par exemple, chaque variable doit être déclarée sur une ligne séparée.
- **Chaque instruction doit respecter strictement la syntaxe AlgoBox**, à l'exception de l'instruction `AFFICHER` qui doit être suivie d'une étoile pour indiquer un retour à la ligne et des instructions `TRACER_POINT` et `TRACER_SEGMENT` qui peuvent être suivies de la couleur à utiliser. (exemple : TRACER_POINT_Bleu, TRACER_POINT_Rouge ou TRACER_POINT_Vert)
- Lors de l'utilisation de l'auto-complétion et des commandes du panneau inférieur, le symbole • représente un champ à compléter : pour passer au champ suivant, il suffit d'utiliser la touche `Tab`.
- Toutes les commandes habituelles d'un éditeur de texte (Copier, Couper, Coller, Défaire, Refaire, Chercher, Remplacer, Indenter, Commenter...) sont disponibles via le menu contextuel qui s'affiche après un clic-droit dans la zone d'édition.
- Il est aussi possible de défnir trois signets en cliquant dans la bande où figurent les numéros de ligne. Le menu contextuel permet alors d'atteindre la position définie par un signet.
- Avant d'exécuter un algorithme édité dans le mode "éditeur de texte", **il est vivement recommandé de vérifier d'abord si le code tapé est bien reconnu par AlgoBox**. Pour cela, il suffit d'utiliser le bouton Vérifier Code : AlgoBox recrée alors l'arbre de code habituel.
- Le changement de mode d'édition ne peut se faire qu'à partir d'un algorithme vierge. Si un algorithme est en cours d'édition, il est demandé à l'utilisateur s'il veut enregistrer son travail avant de changer de mode.
- Toutes les options des menus et toutes les opérations standards (exécuter un algorithme, ouvrir, enregistrer...) restent évidemment disponibles dans le mode "éditeur de texte".
- Le mode d'édition utilisé ("normal" ou "éditeur de texte")
  est enregistré à chaque fermeture du programme afin d'être rétabli par défaut lors du prochain démarrage.

# 11. Le menu "Extension"

Il est possible d'ajouter des commandes à celles disponibles par défaut dans AlgoBox en utilisant une "extension" qui est en fait constituée d'un fichier javascript (format .js).

**ATTENTION : l'écriture d'une extension nécessite de bien connaître le langage javascript et le fonctionnement d'AlgoBox. Toute erreur dans le code de l'extension empêchera le fonctionnement de l'algorithme AlgoBox qui l'utilise.**

Caractéristiques :

- le code d'une extension doit-être écrite en pur javascript sous la forme de **fonctions**.
- pour pouvoir être appelées dans un algorithme AlgoBox, les fonctions de l'extension doivent retourner un nombre ou une chaine (même si ce retour n'a pas en fait d'importance).
  Exemple :
  *Code dans l'extension (en javascript!) :*
  ```
  function MON_EXTENSION()
  {
  ...
  return 1;
  }
  ```
  *Appel de la nouvelle fonction dans un algorithme :*
  ```
  mavariable PREND_LA_VALEUR MONEXTENSION()
  ```
- pour utiliser une extension dans un algorithme AlgoBox, il suffit d'utiliser l'option `"Charger une extension"`. Pour des raisons de portabilité d'un poste à un autre, le chemin complet vers l'extension n'est pas enregistré dans le fichier AlgoBox : il est donc nécessaire de charger l'extension à chaque ouverture de l'algorithme.
- Un algorithme ne peut utiliser qu'une seule extension (il suffit d'inclure toutes les fonctions supplémentaires dans un même fichier javascript).
- Pour écrire ou modifier une extension, un mini-éditeur javascript est accessible via l'option `"Ecrire/modifier une extension"`.




- # Dernière version
  - Version 0.7.2 : 21 juin 2012
- # Contact
  - Cliquer ici pour écrire à l'auteur
- # Pages
  - Présentation
  - Téléchargement
  - Brochures pédagogiques
  - Démo
  - Manuel
  - Initiation
  - Exemples
  - Extensions

- # Liens

    - Texmaker (éditeur LaTeX libre et multi-plateforme)
    - Pstplus (assistants LaTeX/pstricks pour documents mathématiques)

- **AlgoBox** - Copyright 2009/2012 **Pascal Brachet**
- L'auteur est professeur de mathématiques au lycée Bernard Palissy à Agen.
- Ce programme est distribué selon les termes de la licence libre GPL.